



**UNIVERSITY OF
WATERLOO**

University of Waterloo

Department of Mechanical and Mechatronics Engineering

Development of a Layout Algorithm for Plumbing System Visualization

Prepared for

Waterloo Rocketry

Prepared by

Jacob Deery

ID 20650895

4A Mechatronics Engineering

21 September 2020

388 Black Drive
Milton, Ontario, L9T 6R8

21 September 2020

Professor Andrew Kennings
Director, Mechatronics Engineering
Department of Mechanical and Mechatronics Engineering
University of Waterloo
200 University Avenue West
Waterloo, Ontario, N2L 3G1

Dear Professor Kennings,

This report, entitled “Development of a Layout Algorithm for Plumbing System Visualization”, was prepared as my 4A work term report. This is my third and final work report submission.

Waterloo Rocketry is a University of Waterloo student design team with over 50 team members, primarily engineering undergraduate students from various disciplines. The team’s primary objective is to provide students with valuable hands-on learning opportunities through the design, manufacture, and testing of high-powered sounding rockets. I have been a member of this team since September 2016 and the team’s Software Lead since May 2020. As Software Lead, much of my time is spent on the development and maintenance of Topside, the team’s software platform for simulating rocket launch operations. This report describes the design and implementation of one component of Topside: the plumbing visualization algorithm.

This report was written entirely by me and has not received any previous academic credit at this or any other institution. I would like to acknowledge the help of my teammate Wendi Yu, with whom I had many productive discussions about the requirements and approach for this project. I received no other assistance.

Sincerely Yours,

A handwritten signature in black ink, appearing to read 'J. Deery', with a stylized, cursive script.

Jacob Deery
ID 20650895

Table of Contents

List of Figures	iii
Summary	v
1 Introduction	1
1.1 Waterloo Rocketry	1
1.1.1 Fill System	1
1.1.2 Operational Procedures	2
1.1.3 Topside	4
1.2 Objective	4
2 Requirements and Desired Characteristics	5
3 Design	6
3.1 Placement	6
3.1.1 Approach	7
3.1.2 Cost Terms	8
4 Implementation and Refinement	13
5 Discussion and Recommendations	16
6 Conclusions and Recommendations	18
6.1 Conclusions	18
6.2 Recommendations	18
References	19
Appendix A Terminal Graph Source Code	20
Appendix B Cost Term Source Code	22

Appendix C Constraint Source Code	37
Appendix D Optimization Algorithm Source Code	39

List of Figures

Figure 1	A section of Waterloo Rocketry’s fill system	2
Figure 2	A P&ID for Waterloo Rocketry’s fill system in “launch configuration”	3
Figure 3	A P&ID for Waterloo Rocketry’s fill system in “test configuration” .	3
Figure 4	Topside graph representation	6
Figure 5	The terminal graph for the P&ID in Figure 4a	7
Figure 6	The neighbouring distance cost term visualized. In this example, $D = 5$ and $W = 1$	9
Figure 7	The non-neighbouring distance cost term visualized. In this example, $D = 5$ and $W = 1$. Thin black lines between nodes are purely for annotating distance.	10
Figure 8	The right angle cost term visualized. In this example, $W = 1$	11
Figure 9	The horizontal deviation cost term visualized. In this example, $W = 1$	12
Figure 10	A case in which the centroid cost term incentivizes node alignment	12
Figure 11	An example of poor placement with the first implementation of the algorithm. Components G and H are oriented vertically instead of horizontally, and many components deviate slightly from exact right angles.	13
Figure 12	Another example of poor placement, with a higher weight on the right angle cost term. In this example, components are closer to right angles, but are generally not aligned well with each other. The diagram is also much closer to a square than to a landscape aspect ratio.	14
Figure 13	An example of acceptable placement, achieved using the second implementation of the placement algorithm.	15
Figure 14	An example of poor placement. Note that component I, although oriented horizontally, is placed far away from component H and in such a way that routed hoses must cross. This is undesirable.	17

Figure 15	The placement algorithm can correctly place parallel branches with the same number of components.	17
-----------	--	----

Summary

This report documents the design and implementation of an algorithm for visualizing a network of hoses, valves, and other plumbing components.

Drawing inspiration from the printed circuit board design process, the task of plumbing component arrangement was divided into two separate stages: *placement* and *routing*. A trivial approach to routing was deemed satisfactory for the initial stages of development. Various approaches to the more difficult problem of placement were considered, and an optimization-based method was selected.

The placement problem was posed as an optimization problem, and a cost function was constructed to reflect the desired characteristics of the visualization. The algorithm was implemented in software and refined until initial results were deemed to be satisfactory.

The designed algorithm was evaluated on several test cases. Although several shortcomings were identified, the algorithm is considered to be acceptable overall. Several possible modifications and extensions are discussed as areas for future improvement.

1 Introduction

Waterloo Rocketry is a student team at the University of Waterloo focusing on the design, testing, and launch of high-powered sounding rockets [1]. The complex procedures required to conduct a rocket launch have motivated the team to develop a software tool for visualizing, simulating, and validating these procedures: Topside [2]. This report describes the development of an algorithm used in Topside for visualization of the rocket launch plumbing systems.

This report is divided into 6 sections. Section 1 provides background on the Waterloo Rocketry team and introduces the motivation, objectives, and scope of this project. Section 2 outlines the identified requirements and specifications for the algorithm. Section 3 describes the design of the algorithm, and Section 4 describes the implementation and refinement of the proposed design. Section 5 contains evaluation of results and discussion of the strengths, limitations, and suitability of the algorithm, along with recommendations for future development. Finally, Section 6 concludes the report.

1.1 Waterloo Rocketry

Waterloo Rocketry was founded in 2009 and has competed at the Intercollegiate Rocket Engineering Competition since 2011 [1]. The team emphasizes student design and aims to create meaningful learning opportunities for all team members. Towards this end, the team is one of a small subset of schools that has consistently competed with rockets with student-designed engines [1].

1.1.1 Fill System

Waterloo Rocketry primarily develops *hybrid* and *liquid* rockets, which are distinct from solid rockets in their use of at least one liquid propellant. The presence of a liquid oxidizer and/or fuel necessitate increased complexity in the ground systems used to launch the rocket:

a *fill system* is required, comprising a network of hoses, valves, and fittings, along with an electrical system to remotely control these valves [3]. A section of the fill system can be seen in Figure 1. Visible in this photograph are two motorized ball valves (white boxes), three manual ball valves (blue and yellow handles), one pressure gauge, and two pressure transducers, with hoses leading to the oxidizer supply cylinder, the rocket, and emergency vent lines.



Figure 1: A section of Waterloo Rocketry’s fill system

Typically, the fill system is visualized in the form of a Plumbing and Instrumentation Diagram (P&ID). This diagram is analogous to an electrical schematic, in that it provides symbolic representations for all components and depicts the topology and connectivity of the system, but does not attempt to depict exact scale or physical measurements. Two examples of P&IDs can be seen in Figures 2 and 3.

1.1.2 Operational Procedures

In order to achieve a safe and effective launch of the rocket, the team spends considerable time developing *operational procedures*. These procedures detail every action that must be performed as part of the launch process, the member of the launch team that performs the

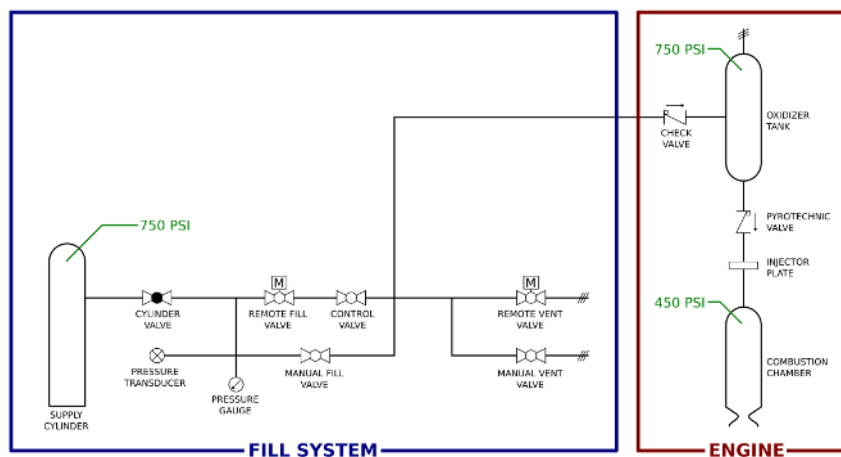


Figure 2: A P&ID for Waterloo Rocketry's fill system in "launch configuration"

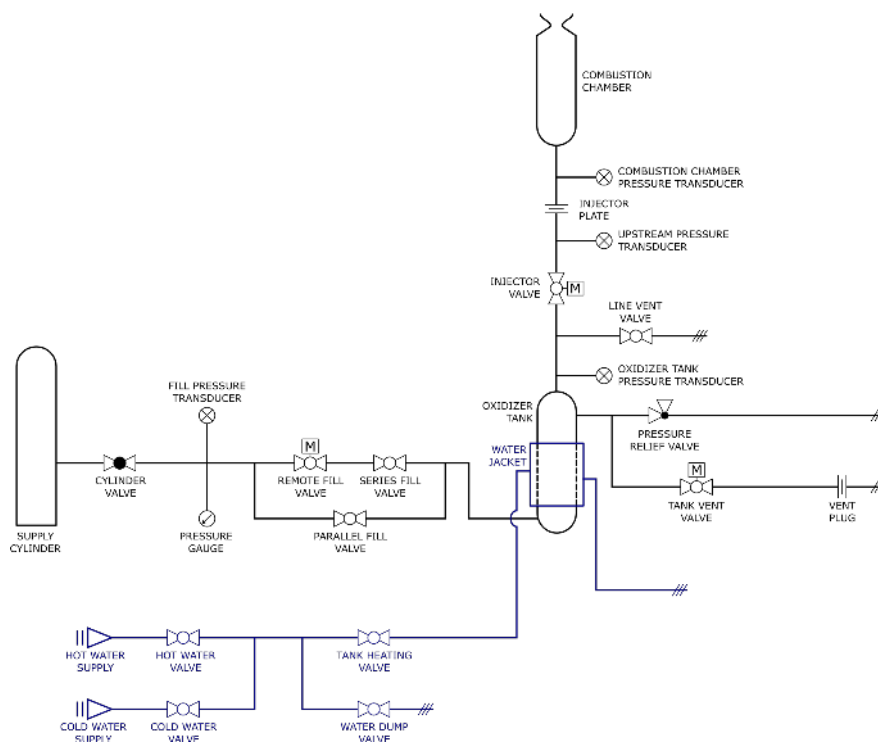


Figure 3: A P&ID for Waterloo Rocketry's fill system in "test configuration"

action, the conditions that must be satisfied before continuing, and the fallback in the event a failure is observed. As a result, these procedures can become quite complex; at the 2019 competition, the nominal procedure for launching the rocket was 156 discrete steps, and there were 6 different off-nominal procedures comprising a total of 107 steps [3].

1.1.3 Topside

The growing complexity of Waterloo Rocketry’s operational procedures has made them increasingly difficult to write, review, and audit. The author of a procedure is required to maintain an image of the state of the system after each step of the procedure, either mentally or by taking notes. This requires substantial mental effort and greatly increases the difficulty of writing or editing a procedure. To address this issue, the team has begun developing a software platform known as Topside. Topside is intended to provide capabilities for simulating the fill system and interacting with it by executing a procedure. The results of opening and closing valves and allowing time to pass can be easily simulated and visualized [2].

1.2 Objective

A key feature of Topside is the ability to visualize the fill system. As Topside is intended to aid the user in understanding and debugging the system, the visualization of the fill system should be easy to understand. The objective of the project described in this report is therefore to develop an algorithm that can, given information about the components and connectivity of the fill system, determine all of the information required for displaying these components in a clear and intuitive manner.

2 Requirements and Desired Characteristics

The first step in the algorithm design process was to identify the desired characteristics of a visualization. After consultation with other members of the Waterloo Rocketry launch operations team, it was agreed that the standard P&ID is clear and intuitive, and can be easily read by both experienced operations personnel and new members to the team. The following characteristics were identified after inspecting several P&IDs from previous launches and engine tests:

1. Component terminals should be **oriented at right angles**. In general, this will produce diagrams that are “neat” and flow naturally.
2. Components should **maintain a minimum separation from each other**. It is important for all components to be sufficiently far apart that the diagram is easy to read and no section is too “busy”.
3. Components that are directly connected should be **close together**. The visualization is more intuitive if components that are close together in the physical fill system are also close together in the diagram.
4. Components that are directly connected should either be **aligned horizontally or vertically** or **at right angles**. As with the first point, this increases the general neatness and readability of the diagram.
5. The diagram should be oriented in **landscape form** - that is, the bounding box should be wider than it is tall.
6. Vent lines (hose terminals open to atmosphere) should be **pointed away from the rest of the system**. This makes the diagram easier to scan, and makes it easy to quickly identify vent locations.

3 Design

After identifying the key requirements of a visualization algorithm, it was observed that requirements 1-5 restrict the position and orientation of components, while requirement 6 restricts the manner in which connecting piping should be drawn. This was recognized to be quite similar to the printed circuit board (PCB) design process, where *placement* (positioning and orienting circuit components) and *routing* (connecting components with traces) are treated as two separate steps. It was decided that the layout algorithm should likewise be divided into placement and routing steps, in order to decrease the complexity of the overall problem. It was also agreed that correct placement was the more important aspect of the problem, and the default routing algorithm of “draw a straight line between the two nodes” would be sufficient for an initial implementation.

3.1 Placement

The first step in developing a placement algorithm was determining the data structure that would be used to store component information. The internal representation of the plumbing system used by Topside is a directed graph, where a set of graph edges represents a component and a graph node represents a connection point between components. A diagram depicting this structure can be seen in Figure 4.

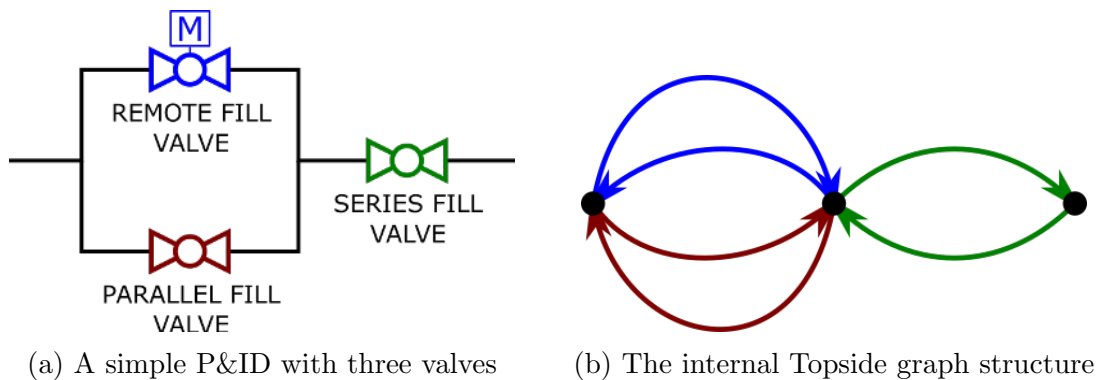


Figure 4: Topside graph representation

Although the internal graph structure is quite suitable for systems modelling and simulation,

it has one main shortcoming: connected components share the same node at their terminal. Looking back at Figure 4, if we wish to replicate the shown P&ID, we will need to place the left terminal of the Remote Fill Valve vertically offset from its connected terminal: the left terminal of the Series Fill Valve. However, with the internal graph structure, this is impossible, as the two terminals are represented by the same node in the graph.

In order to address this problem, we introduce a new graph known as the *terminal graph*. In this graph, each terminal of each component is represented by a separate node, and an additional node is introduced for each junction between components. An example terminal graph for the same example can be seen in Figure 5. Note that each component has two distinct nodes for terminals, and each junction is another node. Source code implementing the terminal graph function is available in Appendix A.

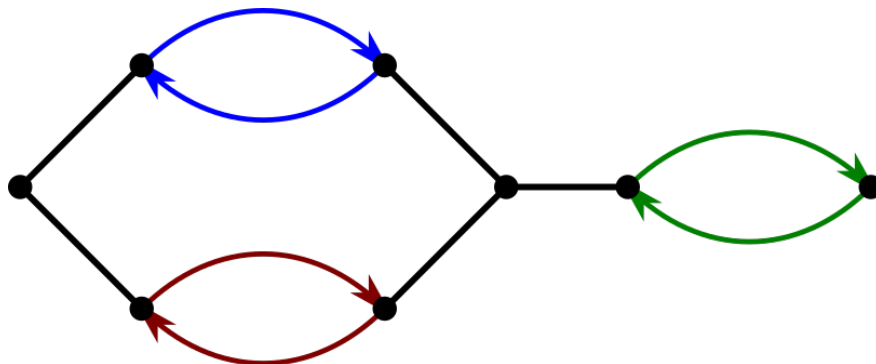


Figure 5: The terminal graph for the P&ID in Figure 4a

By representing the plumbing system with a terminal graph, the placement problem is easy to formulate. Note that in Figure 5, replacing the component edges with symbolic representations would already closely mimic the placement of components in the P&ID. Therefore, we can phrase the placement problem as follows: given a terminal graph T , determine the best placements of the nodes of T so that requirements 1-5 are satisfied.

3.1.1 Approach

Three main approaches were considered for the design of the placement algorithm:

- Traversal: Select a starting node N_0 and place it. Add all the neighbours of N_0 to a queue. While the queue is not empty, remove the next node N_i from the front of the queue, place it, and add all of its neighbours to the back of the queue. When the queue is empty, placement is complete.
- Divide-and-conquer: Break the terminal graph down into smaller “features”, such as cycles and linear segments. Determine how the nodes should be placed in each of these features independently, and then determine how features should be placed relative to each other.
- Optimization: Pose the problem of placing nodes as the minimization of a cost function C over the coordinates of all nodes N_i .

The primary advantages of the traversal and divide-and-conquer approaches were the ease with which they could be understood. However, initial attempts at implementing a proof-of-concept proved quite difficult in both cases. The traversal approach suffered from the requirement to place nodes without knowing what the positions of their neighbours would be required to be; it was common for the initial node placements to result in later node placements that violated one or more of the requirements. The divide-and-conquer approach suffered from a similar problem with early placement of features. Therefore, focus shifted to the implementation of an optimization-based method.

3.1.2 Cost Terms

Intuitively, the structure of the task lends itself well to an optimization approach. The placement of all nodes is determined simultaneously and iteratively, meaning that poor initial placement of a node can be corrected in successive iterations. The requirements can each be converted to cost terms in a cost function and weighted based on their relative importance. Five different cost terms were developed for the cost function; these terms are described below. Source code for all cost terms is available in Appendix B.

The **neighbouring distance** cost term encourages neighbouring nodes to maintain a nominal separation distance. Separation greater or less than the nominal distance is penalized quadratically. This cost term encodes requirement 3. It also encodes the constraint that terminals of a given component must maintain a nominal distance so that the symbolic representation of that component can be placed accurately. For a pair of neighbouring nodes (x_1, y_1) and (x_2, y_2) , the neighbouring distance cost C is defined as follows:

$$\begin{aligned} d &= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \\ C &= W * (D - d)^2 \end{aligned} \tag{1}$$

In Equation 1, W is a scaling factor used for tuning the relative importance of this cost term, and D is the nominal distance between nodes. A visual example of this cost term can be seen in Figure 6. Note that the green node has 0 cost, as it is exactly at D . For the red node, $W * (D - d)^2 = 4$. For the blue node, $W * (D - d)^2 = 9$.

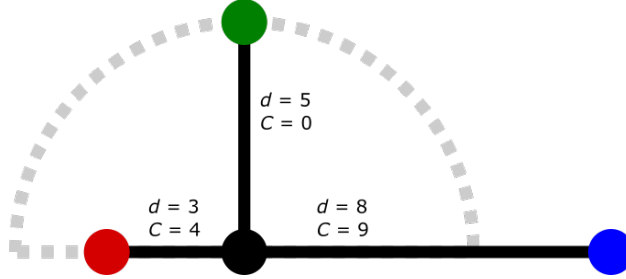


Figure 6: The neighbouring distance cost term visualized. In this example, $D = 5$ and $W = 1$.

The **non-neighbouring distance** cost term encourages non-neighbouring nodes to maintain a minimum separation distance. This cost term encodes requirement 2. It is very similar to the neighbouring distance cost term, and differs only in how the cost is calculated for node distances greater than D . For a pair of neighbouring nodes (x_1, y_1) and (x_2, y_2) , the

neighbouring distance cost C is defined as follows:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$C = \begin{cases} W * (D - d)^2 & \text{if } d < D \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

Note that this cost term is only relevant if nodes start to come too close together; in other cases, such as for two unrelated nodes on opposite sides of the diagram, it will not contribute to the cost function. A visual example of this cost term can be seen in Figure 7. Note that the green node has 0 cost, as it is exactly at D . For the red node, $W * (D - d)^2 = 4$. For the blue node, $d > D$, so $C = 0$.

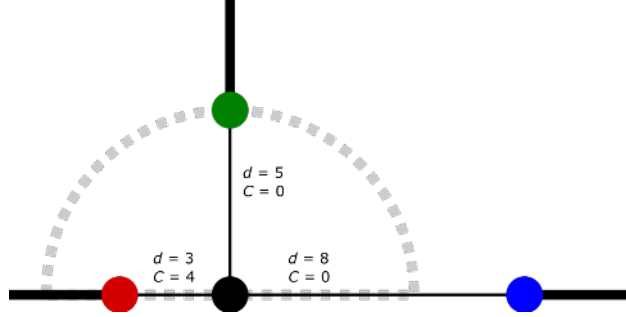


Figure 7: The non-neighbouring distance cost term visualized. In this example, $D = 5$ and $W = 1$. Thin black lines between nodes are purely for annotating distance.

The **right angle deviation** cost term encourages two nodes to be oriented at right angles relative to each other. This cost term encodes requirement 1. For a pair of neighbouring nodes (x_1, y_1) and (x_2, y_2) , the right angle deviation distance cost C is defined as follows:

$$C = W * (x_2 - x_1)^2 * (y_2 - y_1)^2 \quad (3)$$

For this cost term, note that if a component is oriented vertically, $(x_2 - x_1)$ will be 0, and C will therefore be 0; similarly, if the component is oriented horizontally, $(y_2 - y_1)$ will be 0 and C will be 0. We square both differences so that the cost term behaves the same regardless

of if the difference is negative or positive. A visual example of this cost term can be seen in Figure 8. Both the blue and green nodes have no cost, as they are oriented vertically and horizontally relative to the black node. For the red node, $C = 1 * (-5)^2 * 2^2 = 100$.

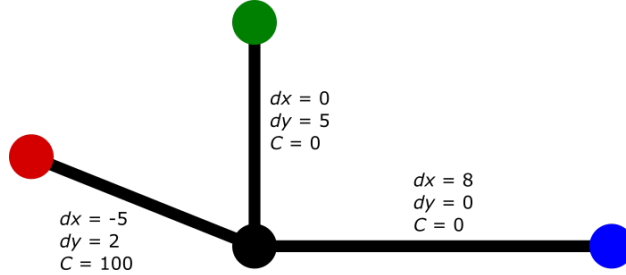


Figure 8: The right angle cost term visualized. In this example, $W = 1$.

The **horizontal deviation** cost term encourages two nodes to be oriented horizontally. This cost term was developed as an alternative to the right angle deviation cost term that additionally incentivizes requirement 5. Horizontally-oriented components are wider than they are tall; therefore, encouraging these components will tend to result in a landscape diagram. For a pair of neighbouring nodes (x_1, y_1) and (x_2, y_2) , the horizontal deviation distance cost C is defined as follows:

$$C = W * (y_2 - y_1)^2 \quad (4)$$

As with the right angle cost, we square $(y_2 - y_1)$ so that the cost term behaves the same regardless of if the difference is negative or positive. A visual example of this cost term can be seen in Figure 9. For the red node, $C = 1 * 2^2 = 4$. For the green node, $C = 1 * 5^2 = 25$. For the blue node, $C = 1 * 0^2 = 0$.

The **centroid deviation** cost term encourages a node to be positioned near the centroid of its neighbours. This cost term was created to encode requirement 4 and incentivize components to line up with each other. An example of a situation in which this cost term would incentivize this behaviour is shown in Figure 10. The black node at the centre of the figure is not located at the centroid of its neighbours. If it were to move to the left, closer

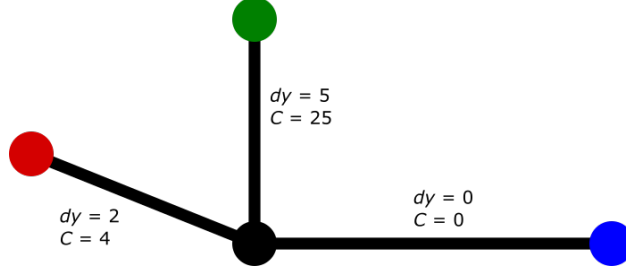


Figure 9: The horizontal deviation cost term visualized. In this example, $W = 1$.

to the centroid, this would cause the blue node to no longer be located at the centroid of its neighbours. Therefore, cost can be minimized most effectively by moving the red node to the right, lining it up with the green node.

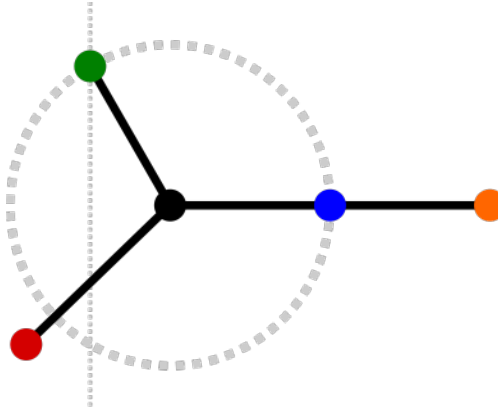


Figure 10: A case in which the centroid cost term incentivizes node alignment

For a given node N and the centroid of its neighbours V , the centroid deviation cost C is defined as follows:

$$C = W * ((V_x - N_x)^2 + (V_y - N_y)^2) \quad (5)$$

Note that for this term, we penalize the absolute x and y deviations rather than the Euclidean distance. This was chosen to simplify the implementation and decrease computational complexity.

4 Implementation and Refinement

As Topside is written in Python, this language was used for implementing the layout algorithm. The NetworkX library [4] was selected for representing the terminal graph, as this library was already used for the Topside internal graph representation. The SciPy library was selected for minimizing the cost function; this library is open-source, well maintained, and has efficient high-quality implementations of many optimization methods [5].

The initial implementation of the optimization algorithm used all cost terms except for the horizontal deviation cost. Unconstrained minimization was performed using the Broyden-Fletcher-Goldfarb-Shanno (BFGS) method [6]. However, these results were unsatisfactory for multiple reasons. Although the right angle cost term encouraged components to be oriented at right angles, it did not impose this as a constraint. As a cost, it still needed to be balanced with the other cost terms. Therefore, the optimal positions for nodes were often slightly off of horizontal or vertical. Increasing the weight (W) on this cost term alleviated this issue somewhat, but did not fully resolve it, and led to diagrams that were undesirable for other reasons. Figures 11 and 12 show two examples of unsatisfactory placement.

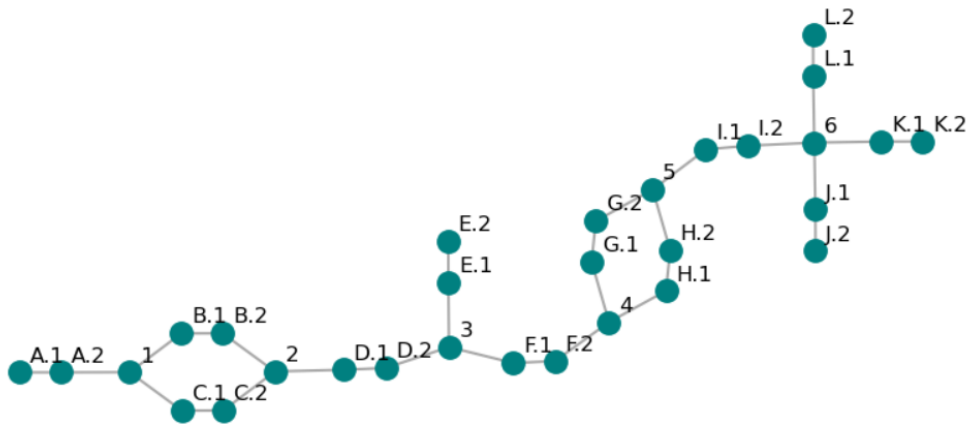


Figure 11: An example of poor placement with the first implementation of the algorithm. Components G and H are oriented vertically instead of horizontally, and many components deviate slightly from exact right angles.

The main problem with this implementation was judged to be the implementation of a

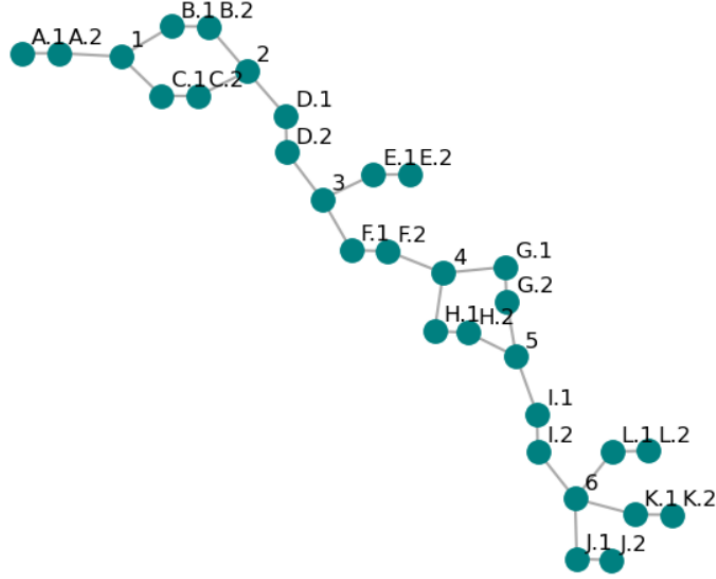


Figure 12: Another example of poor placement, with a higher weight on the right angle cost term. In this example, components are closer to right angles, but are generally not aligned well with each other. The diagram is also much closer to a square than to a landscape aspect ratio.

cost term to incentivize behaviour better enforced as a constraint. Therefore, the second implementation removed the right angle cost term and instead included a **right angle constraint**, defined as follows:

$$(x_2 - x_1) * (y_2 - y_1) = 0 \quad (6)$$

In order to encourage a landscape layout (requirement 5), the horizontal deviation cost term was introduced at this stage. Constrained minimization was performed using the Sequential Least Squares Programming method [7]. For the working example, the results were much more satisfactory. An image of the results can be seen in Figure 13. Source code for the right angle constraint is available in Appendix C, and source code for the complete optimization algorithm is available in Appendix D.



Figure 13: An example of acceptable placement, achieved using the second implementation of the placement algorithm.

5 Discussion and Recommendations

For the example in Figure 13, all of the requirements have been satisfied.

1. **Component terminals should be oriented at right angles.** This was satisfied through the implementation of a right angle constraint. For this example, all components are oriented horizontally.
2. **Components should maintain a minimum separation from each other.** All components in this example are evenly spaced with no overlap. The diagram does not look crowded.
3. **Components that are directly connected should be close together.** No two connected components seem to be too far apart; connected components such as G and H are close together.
4. **Components that are directly connected should either be aligned horizontally or vertically or at right angles.** This is also present for all components; for example, components B and C are vertically aligned, and components A and D are horizontally aligned.
5. **The diagram should be oriented in landscape form.** The diagram is clearly wider than it is tall.
6. **Vent lines should be pointed away from the rest of the system.** Even though this was a requirement for the routing algorithm, not placement, the placement of the nodes in this example has made this easy. Note that vent nodes E.2, J.2, K2, and L.2 are all oriented away from the system.

However, although the algorithm is satisfactory for this example, it is not difficult to construct a plumbing graph for which it fails. An example can be seen in Figure 14.

This appears to be a problem specifically when two parallel branches are unbalanced in

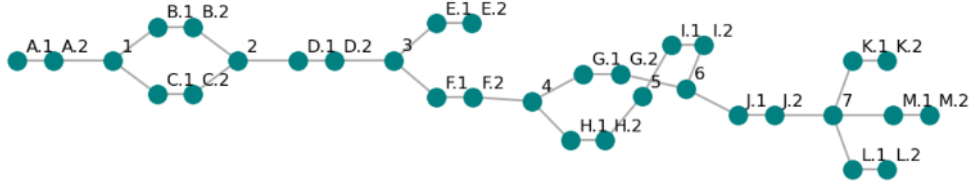


Figure 14: An example of poor placement. Note that component I, although oriented horizontally, is placed far away from component H and in such a way that routed hoses must cross. This is undesirable.

number of components. Inserting a second component to the other branch results again in satisfactory placement, as seen in Figure 15. The exact cause of this issue is still uncertain.

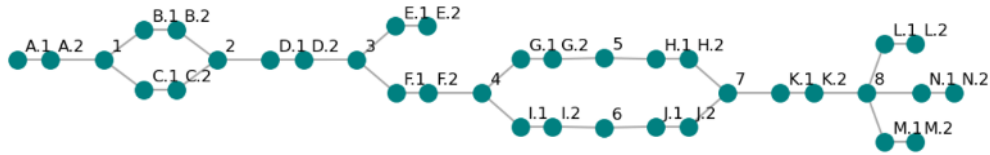


Figure 15: The placement algorithm can correctly place parallel branches with the same number of components.

Despite this shortcoming, the algorithm was determined to be acceptable as the first piece of the Topside plumbing visualization pipeline. The failure observed in the previous figures is a localized failure; it does not result in the entire visualization becoming unstable and unusable. Even in the event that a fill system contains this feature and the visualization is less clear, the overall diagram is still clear and intuitive enough to be usable.

6 Conclusions and Recommendations

Having judged the initial algorithm acceptable, we conclude with recommendations for further improvement.

6.1 Conclusions

This report has described the design and implementation of an algorithm used for visualization of valves, hoses, and other components of a plumbing network. The optimization-based approach initially implemented was judged to be satisfactory for a first version, but with shortcomings that should be addressed in subsequent design iterations. The design of the algorithm makes it quite easy to extend and modify.

6.2 Recommendations

The structure of an optimization-based algorithm is conducive to experimentation and expansion. It is easy to experiment with new functionality or alter results by tuning weighting coefficients or adding new cost terms. There are several areas for potential improvement. First, the issue seen in Figure 14 should be addressed. This may be the result of poor cost term tuning, or it may represent a more fundamental issue with the choice of cost terms. An additional area for investigation is a hybrid algorithm that combines aspects of the divide-and-conquer approach with the optimization approach. It may be possible to lay out individual features using a simpler algorithm, and then place features relative to each other with an optimization approach. Finally, although not discussed in great detail in this report, the routing algorithm could be improved from the simple point-to-point algorithm used in these examples. In order to more closely replicate the example P&IDs, routed connections should also be at right angles, rather than the sloped connections seen in Figures 12-15.

References

- [1] *Waterloo rocketry*, Waterloo Rocketry, 2020. [Online]. Available: <http://www.waterloorocketry.com> (visited on 09/20/2020).
- [2] *Topside*, <https://github.com/waterloo-rocketry/topside>, Waterloo Rocketry, 2020.
- [3] N. Christopher, T. Cojocar, J. Deery, R. Jiang, D. Kletke, R. Kobets, S. Kong, S. Martin, Z. Mears, A. Mihaila, A. Morrison, V. Rajkumar, J. Robinson, K. Tam, T. Tang, and E. Yang, “Shark of the sky hybrid rocket,” in *Spaceport America Cup Conference*, 2019.
- [4] A. Hagberg, D. Schult, and P. Swart, *Networkx*, <https://github.com/networkx/networkx>, 2020.
- [5] *Scipy*, <https://github.com/scipy/scipy>, SciPy Developers, 2020.
- [6] J. Nocedal and S. J. Wright, *Numerical Optimization*. Springer New York, 2006, p. 136.
- [7] D. Kraft, “A software package for sequential quadratic programming,” DLR German Aerospace Center - Institute for Flight Mechanics, Koln, Germany, Tech. Rep. DFVLR-FB 88-28, 1988.

Appendix A Terminal Graph Source Code

```
1 import networkx as nx
2
3
4 def terminal_graph(plumbing_engine):
5     """
6     Create an augmented graph suitable for visualization.
7
8     The terminal graph T is created by the following process, for a
9     plumbing engine P with graph G:
10         1. For each non-atm node in G:
11             a) Create a node of the same name in T.
12         2. For each component C in P:
13             a) For each node n in C:
14                 i. Create a node C_n in T, where the node name of C_n is
15                    <name of C>.<name of n>.
16                 ii. Find the node v in G to which n is mapped. This is
17                     defined as P.mapping[C][n].
18                 iii. Create an edge in T from C_n to the node in T with
19                     the same name as v.
20
21     Parameters
22     -----
23
24     plumbing_engine: topside.PlumbingEngine
25
26     Returns
27     -----
28
29     t: networkx.Graph
30     """
31     t = nx.Graph()
```

```

32
33     for name, c in plumbing_engine.component_dict.items():
34         for edge in c.component_graph.edges:
35             (n1, n2, edge_id) = edge
36             t.add_edge(name + '.' + str(n1), name + '.' + str(n2))
37         for node in c.component_graph.nodes:
38             if not plumbing_engine.mapping[name][node] == 'atm':
39                 t.add_edge(name + '.' + str(node), plumbing_engine.mapping
[ name ][ node ])
40
41     return t

```

Appendix B Cost Term Source Code

```
1 import numpy as np
2
3
4 class NeighboringDistance():
5     """
6     Encourages a nominal distance between neighboring nodes.
7
8     For two nodes [x1, y1] and [x2, y2] with nominal separation D, the
9     neighboring distance cost C is defined as:
10
11         
$$C = W * (D - \sqrt{(x2 - x1)^2 + (y2 - y1)^2})^2$$

12
13     where W is a constant used to scale the relative "importance" of
14     this cost term.
15     """
16
17     def __init__(self, g, node_indices, node_component_neighbors, settings
18     , internal):
19         """
20         Initialize the cost term.
21
22         Parameters
23         -----
24
25         g: graph
26             The graph for which this cost term will be evaluated.
27             Typically, this is a NetworkX graph, but it can be any type
28             with a .neighbors(n) method returning the neighbors of node
29             n. Calling this directly with a PlumbingEngine graph may
30             lead to unexpected results; it is best to use a terminal
31             graph instead (topside.terminal_graph).
```

```

31
32     node_indices: dict
33         Dict where node_indices[n] is the index i of node n in the
34         cost vector.
35
36     node_component_neighbors: dict
37         Dict where, for a given node n, node_component_neighbors[n]
38         is a list of all nodes in g that are both neighbors of n
39         and in the same component as n.
40
41     settings: OptimizerSettings
42         This can be any type that provides the same attributes as
43         topside.visualization.optimization.OptimizerSettings.
44
45     internal: bool
46         if True, this cost term penalizes only deviation from
47         nominal distance within a component. If False, this cost
48         term instead penalizes deviation from nominal distance
49         between two nodes that are not part of the same component.
50     """
51
52     self.num_nodes = len(node_indices)
53
54     self.mask = np.ones((self.num_nodes, self.num_nodes))
55
56     for node in g:
57         i = node_indices[node]
58         for neighbor in g.neighbors(node):
59             j = node_indices[neighbor]
60             if internal is (neighbor in node_component_neighbors[node
61 ]):
62                 self.mask[i, j] = 0
63                 self.mask[j, i] = 0

```

```

63
64     if internal:
65         self.nominal_dist = settings.nominal_dist_internal
66         self.weight = settings.internal_weight
67     else:
68         self.nominal_dist = settings.nominal_dist_neighbors
69         self.weight = settings.neighbors_weight
70
71     def evaluate(self, costargs):
72         """
73         Evaluate the cost term and return a tuple of (cost, gradient).
74
75         Arguments:
76
77         costargs: dict
78             Expected to contain:
79             - 'deltas': N x N x 2 numpy array, where:
80                 deltas[i, j, :] == [xj - xi, yj - yi]
81             - 'norms': N x N numpy array, where:
82                 norms[i, j] == sqrt[(xj - xi)^2 + (yj - yi)^2]
83         """
84         cost = 0
85         grad = np.zeros(self.num_nodes * 2)
86
87         norms = np.ma.masked_array(costargs['norms'], mask=self.mask)
88
89         cost_matrix = self.weight * (self.nominal_dist - norms) ** 2
90         cost += np.sum(cost_matrix.filled(0))
91
92         grad_common = self.weight * (norms - self.nominal_dist) * (4 /
norms)
93
94         grad_matrix = costargs['deltas'] * grad_common[:, :, None]
95         grad += np.reshape(np.sum(grad_matrix.filled(0), axis=1), grad.

```

```

shape)

95
96         return (cost, grad)
97
98
99 class NonNeighboringDistance:
100     """
101     Encourages a minimum distance between non-neighboring nodes.
102
103     For two nodes [x1, y1] and [x2, y2] with minimum separation D, the
104     non-neighboring distance cost C can be calculated with the following
105     pseudocode algorithm:
106
107         d = sqrt((x2 - x1)^2 + (y2 - y1)^2)
108
109         if d < D:
110             C = W * (D - d)^2
111         else:
112             C = 0
113
114     where W is a constant used to scale the relative "importance" of
115     this cost term.
116     """
117
118     def __init__(self, g, node_indices, node_component_neighbors, settings
119 ):
120         """
121         Initialize the cost term.
122
123         Parameters
124         -----
125
126         g: graph

```

```

126         The graph for which this cost term will be evaluated.
127         Typically, this is a NetworkX graph, but it can be any type
128         with a .neighbors(n) method returning the neighbors of node
129         n. Calling this directly with a PlumbingEngine graph may
130         lead to unexpected results; it is best to use a terminal
131         graph instead (topside.terminal_graph).
132
133     node_indices: dict
134         Dict where node_indices[n] is the index i of node n in the
135         cost vector.
136
137     node_component_neighbors: dict
138         Dict where, for a given node n, node_component_neighbors[n]
139         is a list of all nodes in g that are both neighbors of n
140         and in the same component as n.
141
142     settings: OptimizerSettings
143         This can be any type that provides the same attributes as
144         topside.visualization.optimization.OptimizerSettings.
145     """
146     self.num_nodes = len(node_indices)
147
148     self.mask = np.identity(self.num_nodes)
149
150     for node in g:
151         i = node_indices[node]
152
153         for neighbor in g.neighbors(node):
154             j = node_indices[neighbor]
155             self.mask[i, j] = 1
156             self.mask[j, i] = 1
157
158     self.weight = settings.others_weight

```

```

159         self.minimum_dist = settings.minimum_dist_others
160
161     def evaluate(self, costargs):
162         """
163         Evaluate the cost term and return a tuple of (cost, gradient).
164
165         Arguments:
166
167         costargs: dict
168             Expected to contain:
169             - 'deltas': N x N x 2 numpy array, where:
170                 deltas[i, j, :] == [xj - xi, yj - yi]
171             - 'norms': N x N numpy array, where:
172                 norms[i, j] == sqrt((xj - xi)^2 + (yj - yi)^2)
173         """
174         cost = 0
175         grad = np.zeros(self.num_nodes * 2)
176
177         nodes_to_ignore = (costargs['norms'] >= self.minimum_dist)
178         mask = np.logical_or(self.mask, nodes_to_ignore)
179
180         masked_norms = np.ma.masked_array(costargs['norms'], mask=mask)
181
182         cost_matrix = self.weight * (self.minimum_dist - masked_norms) **
2
183         cost += np.sum(cost_matrix.filled(0))
184
185         grad_common_term = self.weight * (masked_norms - self.minimum_dist
186         ) * (4 / masked_norms)
187
188         grad_matrix = costargs['deltas'] * grad_common_term[:, :, None]
189
190         grad += np.reshape(np.sum(grad_matrix.filled(0), axis=1), grad.
191         shape)

```

```

189
190         return (cost, grad)
191
192
193 class CentroidDeviation:
194     """
195     Encourages a node to remain close to the centroid of its neighbors.
196
197     The centroid deviation cost term C can be calculated with the
198     following pseudocode algorithm:
199
200     if num_neighbors(n) < 2:
201         C = 0
202     else:
203         v = centroid(neighbors(n))
204         C = W * ((vx - nx)^2 + (vy - ny)^2)
205
206     where W is a constant used to scale the relative "importance" of
207     this cost term.
208     """
209
210     def __init__(self, g, node_indices, node_component_neighbors, settings
211 ):
212         """
213         Initialize the cost term.
214
215         Parameters
216         -----
217
218         g: graph
219             The graph for which this cost term will be evaluated.
220             Typically, this is a NetworkX graph, but it can be any type
221             with a .neighbors(n) method returning the neighbors of node

```

```

221         n. Calling this directly with a PlumbingEngine graph may
222         lead to unexpected results; it is best to use a terminal
223         graph instead (topside.terminal_graph).
224
225     node_indices: dict
226         Dict where node_indices[n] is the index i of node n in the
227         cost vector.
228
229     node_component_neighbors: dict
230         Dict where, for a given node n, node_component_neighbors[n]
231         is a list of all nodes in g that are both neighbors of n
232         and in the same component as n.
233
234     settings: OptimizerSettings
235         This can be any type that provides the same attributes as
236         topside.visualization.optimization.OptimizerSettings.
237     """
238     self.num_nodes = len(node_indices)
239
240     self.cost_mask = np.ones((self.num_nodes, self.num_nodes, 2))
241     self.grad_mask = np.ones((self.num_nodes, self.num_nodes, 2))
242     self.num_neighbors = np.ones((self.num_nodes, 1))
243
244     for node in g:
245         i = node_indices[node]
246         neighbors = list(g.neighbors(node))
247         if len(neighbors) > 1:
248             self.grad_mask[i, i] = 0
249             self.num_neighbors[i] = len(neighbors)
250             for neighbor in neighbors:
251                 j = node_indices[neighbor]
252                 self.cost_mask[i, j] = 0
253                 self.grad_mask[j, i] = 0

```

```

254
255     self.weight = settings.centroid_deviation_weight
256
257     grad_coeffs_diag = self.weight * 2 * np.identity(self.num_nodes)
258
259     grad_coeffs_off_diag = self.weight * -2 * \
260         np.ones((self.num_nodes, self.num_nodes, 2)) / self.
num_neighbors
261     np.fill_diagonal(grad_coeffs_off_diag[:, :, 0], 0)
262     np.fill_diagonal(grad_coeffs_off_diag[:, :, 1], 0)
263
264     self.grad_coeffs = grad_coeffs_off_diag + grad_coeffs_diag[:, :,
None]
265
266     def evaluate(self, costargs):
267         """
268         Evaluate the cost term and return a tuple of (cost, gradient).
269
270         Arguments:
271
272         costargs: dict
273             Expected to contain:
274             - 'deltas': N x N x 2 numpy array, where:
275                 deltas[i, j, :] == [xj - xi, yj - yi]
276             - 'norms': N x N numpy array, where:
277                 norms[i, j] == sqrt[(xj - xi)^2 + (yj - yi)^2]
278         """
279         cost = 0
280         grad = np.zeros(self.num_nodes * 2)
281
282         masked_deltas = np.ma.masked_array(costargs['deltas'], mask=self.
cost_mask)
283         centroid_deviations = np.sum(masked_deltas, axis=1) / self.

```

```

num_neighbors

284
285     cost_matrix = self.weight * centroid_deviations ** 2
286     cost += np.sum(cost_matrix.filled(0))
287
288     reshaped = np.reshape(centroid_deviations, (1, self.num_nodes, 2))
289     repeated = np.repeat(reshaped, self.num_nodes, axis=0)
290
291     grad_deviations = np.ma.masked_array(repeated, mask=self.grad_mask
)
292
293     grad_matrix = self.grad_coeffs * grad_deviations
294     grad += np.reshape(np.sum(grad_matrix.filled(0), axis=1), grad.
shape)
295
296     return (cost, grad)
297
298
299 class RightAngleDeviation:
300     """
301     Encourages a component to be oriented horizontally or vertically.
302
303     The right angle deviation cost term C is defined as follows:
304
305     
$$C = W * (x_2 - x_1)^2 * (y_2 - y_1)^2$$

306
307     where W is a constant used to scale the relative "importance" of
308     this cost term.
309     """
310
311     def __init__(self, g, node_indices, node_component_neighbors, settings
):
312         """

```

```

313     Initialize the cost term.
314
315     Parameters
316     -----
317
318     g: graph
319         The graph for which this cost term will be evaluated.
320         Typically, this is a NetworkX graph, but it can be any type
321         with a .neighbors(n) method returning the neighbors of node
322         n. Calling this directly with a PlumbingEngine graph may
323         lead to unexpected results; it is best to use a terminal
324         graph instead (topside.terminal_graph).
325
326     node_indices: dict
327         Dict where node_indices[n] is the index i of node n in the
328         cost vector.
329
330     node_component_neighbors: dict
331         Dict where, for a given node n, node_component_neighbors[n]
332         is a list of all nodes in g that are both neighbors of n
333         and in the same component as n.
334
335     settings: OptimizerSettings
336         This can be any type that provides the same attributes as
337         topside.visualization.optimization.OptimizerSettings.
338     """
339     self.num_nodes = len(node_indices)
340
341     self.mask = np.ones((self.num_nodes, self.num_nodes, 2))
342     for node in g:
343         i = node_indices[node]
344         for neighbor in node_component_neighbors[node]:
345             j = node_indices[neighbor]

```

```

346         self.mask[i, j] = 0
347         self.mask[j, i] = 0
348
349     self.weight = settings.right_angle_weight
350
351     def evaluate(self, costargs):
352         """
353         Evaluate the cost term and return a tuple of (cost, gradient).
354
355         Arguments:
356
357         costargs: dict
358             Expected to contain:
359             - 'deltas': N x N x 2 numpy array, where:
360                 deltas[i, j, :] == [xj - xi, yj - yi]
361             - 'norms': N x N numpy array, where:
362                 norms[i, j] == sqrt[(xj - xi)^2 + (yj - yi)^2]
363         """
364         cost = 0
365         grad = np.zeros(self.num_nodes * 2)
366
367         internal_deltas = np.ma.masked_array(costargs['deltas'], mask=self
368 .mask)
369
370         dxdy = np.product(internal_deltas, axis=2)
371
372         cost_matrix = self.weight * dxdy ** 2
373         cost += np.sum(cost_matrix.filled(0))
374
375         grad_common = self.weight * 4 * dxdy[:, :, None]
376         grad_matrix = grad_common * np.flip(internal_deltas, axis=2)
377         grad += np.reshape(np.sum(grad_matrix.filled(0), axis=1), grad.
378 shape)

```

```

377
378         return (cost, grad)
379
380
381 class HorizontalDeviation:
382     """
383     Encourages a component to be oriented horizontally.
384
385     The right angle deviation cost term C is defined as follows:
386
387     
$$C = W * (y2 - y1)^2$$

388
389     where W is a constant used to scale the relative "importance" of
390     this cost term.
391     """
392
393     def __init__(self, g, node_indices, node_component_neighbors, settings
394 ):
395         """
396         Initialize the cost term.
397
398         Parameters
399         -----
400
401         g: graph
402             The graph for which this cost term will be evaluated.
403             Typically, this is a NetworkX graph, but it can be any type
404             with a .neighbors(n) method returning the neighbors of node
405             n. Calling this directly with a PlumbingEngine graph may
406             lead to unexpected results; it is best to use a terminal
407             graph instead (topside.terminal_graph).
408
409         node_indices: dict

```

```

409         Dict where node_indices[n] is the index i of node n in the
410         cost vector.
411
412     node_component_neighbors: dict
413         Dict where, for a given node n, node_component_neighbors[n]
414         is a list of all nodes in g that are both neighbors of n
415         and in the same component as n.
416
417     settings: OptimizerSettings
418         This can be any type that provides the same attributes as
419         topside.visualization.optimization.OptimizerSettings.
420     """
421     self.num_nodes = len(node_indices)
422
423     self.mask = np.ones((self.num_nodes, self.num_nodes))
424     for node in g:
425         i = node_indices[node]
426         for neighbor in node_component_neighbors[node]:
427             j = node_indices[neighbor]
428             self.mask[i, j] = 0
429             self.mask[j, i] = 0
430
431     self.weight = settings.horizontal_weight
432
433     def evaluate(self, costargs):
434         """
435         Evaluate the cost term and return a tuple of (cost, gradient).
436
437         Arguments:
438
439         costargs: dict
440             Expected to contain:
441             - 'deltas': N x N x 2 numpy array, where:

```

```

442         deltas[i, j, :] == [xj - xi, yj - yi]
443     - 'norms': N x N numpy array, where:
444         norms[i, j] == sqrt[(xj - xi)^2 + (yj - yi)^2]
445     """
446     cost = 0
447     grad = np.zeros(self.num_nodes * 2)
448
449     delta_y = costargs['deltas'][:, :, 1]
450
451     internal_delta_y = np.ma.masked_array(delta_y, mask=self.mask)
452
453     cost_matrix = self.weight * internal_delta_y ** 2
454     cost += np.sum(cost_matrix.filled(0))
455
456     grad_matrix = np.ma.masked_array(np.zeros((self.num_nodes, self.
num_nodes, 2)))
457     grad_matrix[:, :, 1] = self.weight * 4 * internal_delta_y
458     grad += np.reshape(np.sum(grad_matrix.filled(0), axis=1), grad.
shape)
459
460     return (cost, grad)

```

Appendix C Constraint Source Code

```
1 import numpy as np
2
3
4 def right_angle_cons_f(i, j, x):
5     return (x[2*i] - x[2*j]) * (x[2*i+1] - x[2*j+1])
6
7
8 def right_angle_cons_j(i, j, x):
9     grad = np.zeros((1, len(x)))
10
11     ix = 2*i
12     iy = 2*i+1
13     jx = 2*j
14     jy = 2*j+1
15
16     dx = x[ix] - x[jx]
17     dy = x[iy] - x[jy]
18
19     grad[0, ix] = dy
20     grad[0, iy] = dx
21     grad[0, jx] = -dy
22     grad[0, jy] = -dx
23
24     return grad
25
26
27 # NOTE(jacob): The optimization method we're using right now doesn't
28 # use the Hessian of the constraint, but we still implement it so that
29 # we can easily switch optimization methods if desired.
30 def right_angle_cons_h(i, j, x, v):
31     hess = np.zeros((len(x), len(x)))
```

```
32
33     ix = 2*i
34     iy = 2*i+1
35     jx = 2*j
36     jy = 2*j+1
37
38     hess[ix, iy] = 1
39     hess[iy, ix] = 1
40
41     hess[ix, jy] = -1
42     hess[jy, ix] = -1
43
44     hess[jx, iy] = -1
45     hess[iy, jx] = -1
46
47     hess[jx, jy] = 1
48     hess[jy, jx] = 1
49
50     return hess
```

Appendix D Optimization Algorithm Source Code

```
1 from dataclasses import dataclass
2 from functools import partial
3
4 import numpy as np
5 from scipy.optimize import minimize, NonlinearConstraint
6
7 import topside as top
8
9
10 @dataclass
11 class OptimizerSettings:
12     nominal_dist_neighbors: float = 7
13     neighbors_weight: float = 10
14
15     nominal_dist_internal: float = 4
16     internal_weight: float = 50
17
18     minimum_dist_others: float = 10
19     others_weight: float = 1
20
21     right_angle_weight: float = 0
22     horizontal_weight: float = 0
23
24     centroid_deviation_weight: float = 15
25
26
27 def make_cost_terms(g, node_indices, neighbors, settings):
28     c1 = top.NeighboringDistance(g, node_indices, neighbors, settings,
29                                 internal=True)
30
31     c2 = top.NeighboringDistance(g, node_indices, neighbors, settings,
32                                 internal=False)
```

```

30     c3 = top.NonNeighboringDistance(g, node_indices, neighbors, settings)
31     c4 = top.CentroidDeviation(g, node_indices, neighbors, settings)
32     c5 = top.RightAngleDeviation(g, node_indices, neighbors, settings)
33     c6 = top.HorizontalDeviation(g, node_indices, neighbors, settings)
34
35     return [c1, c2, c3, c4, c5, c6]
36
37
38 def make_constraints(components, node_indices):
39     # TODO(jacob): Consider reformulating this to only create one
40     # constraint that covers every component instead of having N
41     # constraints for N components.
42     constraints = []
43     for cnodes in components:
44         i = node_indices[cnodes[0]]
45         j = node_indices[cnodes[1]]
46
47         cons_f = partial(top.right_angle_cons_f, i, j)
48         cons_j = partial(top.right_angle_cons_j, i, j)
49         cons_h = partial(top.right_angle_cons_h, i, j)
50
51         cons = NonlinearConstraint(cons_f, 0, 0, jac=cons_j, hess=cons_h)
52
53         constraints.append(cons)
54
55     return constraints
56
57
58 def make_costargs(x):
59     xr = np.reshape(x, (-1, 2))
60     deltas = xr[:, None, :] - xr[None, :, :]
61     norms = np.linalg.norm(deltas, axis=2)
62

```

```

63     costargs = {
64         'deltas': deltas,
65         'norms': norms
66     }
67
68     return costargs
69
70
71 def cost_fn(x, cost_terms):
72     # Pre-calculate deltas and norms to avoid repeated calculation
73     # between cost terms.
74     costargs = make_costargs(x)
75
76     costs = []
77     grads = []
78
79     for ct in cost_terms:
80         cost, grad = ct.evaluate(costargs)
81         costs.append(cost)
82         grads.append(grad)
83
84     cost = sum(costs)
85     grad = sum(grads)
86
87     return (cost, grad)
88
89
90 def pos_dict_to_vector(pos, node_indices):
91     a = np.zeros((len(pos) * 2, 1))
92     for n, i in node_indices.items():
93         a[2*i, 0] = pos[n][0]
94         a[2*i+1, 0] = pos[n][1]
95     return a

```

```

96
97
98 def vector_to_pos_dict(v, node_indices):
99     if v.ndim == 1:
100         v = np.reshape(v, (-1, 1))
101     return {n: np.array([v[2*i, 0], v[2*i+1, 0]]) for n, i in node_indices
102             .items()}
103
104 def make_initial_pos(num_nodes):
105     # TODO(jacob): Investigate smarter initialization strategies.
106     initial_pos = np.zeros((num_nodes*2, 1))
107     for i in range(num_nodes):
108         initial_pos[2*i] = i
109         initial_pos[2*i+1] = i / 2
110
111     return initial_pos
112
113
114 def layout_plumbing_engine(plumbing_engine):
115     """
116     Given a plumbing engine, determine the best placement of components.
117
118     Parameters
119     -----
120
121     plumbing_engine: topside.PlumbingEngine
122
123     Returns
124     -----
125
126     pos: dict
127         dict with keys corresponding to the nodes in the terminal graph

```

```

128         of plumbing_engine and values corresponding to the x-y point
129         that the node should be placed at.
130     """
131     t = top.terminal_graph(plumbing_engine)
132     components = top.component_nodes(plumbing_engine)
133
134     node_indices = {n: i for i, n in enumerate(t.nodes)}
135
136     neighbors = {n: [] for n in t.nodes}
137     for cnodes in components:
138         for n in cnodes:
139             neighbors[n] = [v for v in t.neighbors(n) if v in cnodes]
140
141     initial_pos = make_initial_pos(t.order())
142     constraints = make_constraints(components, node_indices)
143
144     stage_2_settings = OptimizerSettings()
145     stage_2_cost_terms = make_cost_terms(
146         t, node_indices, neighbors, stage_2_settings)
147     stage_2_args = (stage_2_cost_terms)
148
149     fine_tuning_res = minimize(cost_fn, initial_pos, jac=True, method='
SLSQP',
150                               constraints=constraints, args=stage_2_args,
151                               options={'maxiter': 200})
152     print(fine_tuning_res)
153
154     pos = top.vector_to_pos_dict(fine_tuning_res.x, node_indices)
155
156     return pos

```